



综合实验： 流水线RISC-V处理器

2025年秋

内容概要

- 处理器实验概要
- RISC-V五级流水线回顾
- 进阶功能提示
- 监控程序运行流程
- 常见问题讨论

我们要做什么？

奋战三星期
做台计算机

我们要做什么？

□ 实验任务

- 支持指令流水的CPU
- 使用基本存储和扩展存储以及输入/输出
- 进行扩展

□ 实验目标

- 60分：完成个人实验+流水线处理器运行监控程序的基础版本
- 60分-100分：鼓励额外功能性能特点，包括但不限于中断、虚拟地址、外设驱动、ucore执行、特色应用、动态分支预测、缓存等
- 100分：实现Cache且能运行监控版本3、能够运行ucore、同等水平

时间安排

□ 重要时间节点

- 综合实验讨论：11月13日（展示流水线数据通路的设计）
- 实验检查：12月3日~12月10日（只能检查一次）
- 期中考试（实验测试）：12月11日
- 实验报告上交：12月12日
- 实验分组答辩：12月18日

□ 答疑及实验室开放

- 大实验期间每周四下午14:00~16:30
- 地点：计算机系教学实验室（自强科技楼4号楼12层）
- 实验室开放地点：待定

实验成果

□ 完整的实验报告

- 实验目标
- 实验内容
- 实验展示
- 实验心得体会

□ 完整的源代码

- 设计框图
- 指令流程表
- Verilog代码
- 其他代码

大实验报告要求

□ 实验目标概述

□ 主要模块设计

- 硬件：Datapath、Controller、Memory、I/O
- 软件：对系统软件的修改、应用软件

□ 主要模块实现

□ 实验成果展示

□ 实验心得和体会

□ 提交材料

- 实验报告
- 视频材料
- 完整源代码及相关说明

具体实验要求

1. 能够支持监控程序基本版本用到的 RV32I 指令集。
2. 每一个小组还需要实现指定给本小组的额外三条指令。布置的指令均来自 RISC-V B 扩展。
3. 具有内存（SRAM）访问功能，能够满足监控程序数据与代码的存储需求。
4. 利用串口实现计算机的输入输出模块，能够支持监控程序与 PC 的相互通信。
5. 作为提高要求，实现中断处理机制，对串口产生的中断信号，运行中断处理程序接收数据。
6. 作为提高要求，支持虚拟内存管理，分离用户程序和监控程序内核的地址空间。

指令集的实现—ISA 理解

- 手册是权威标准！
- 19+x条指令如何编码？
指令的语义？
- 每条指令在各阶段/
状态要完成的功能？
- 监控程序如何使用这些指令
 - 尤其注意访存指令
- 地址空间划分规则

The RISC-V Instruction Set Manual
Volume I: Unprivileged ISA
Document Version 20191213

Editors: Andrew Waterman¹, Krste Asanović^{1,2}
¹SiFive Inc.,
²CS Division, EECS Department, University of California, Berkeley
andrew@sifive.com, krste@berkeley.edu
December 13, 2019

The RISC-V Instruction Set Manual
Volume II: Privileged Architecture
Document Version 20211203

Editors: Andrew Waterman¹, Krste Asanović^{1,2}, John Hauser
¹SiFive Inc.,
²CS Division, EECS Department, University of California, Berkeley
andrew@sifive.com, krste@berkeley.edu, jh.riscv@jhauser.us
December 4, 2021

基本步骤

□ 确定目标

- 工作计划
- 组内分工

□ 概要设计

- 数据通路设计
- 模块设计
- 流水段设计
- 控制信号设计

□ 详细设计

- 各模块编码实现

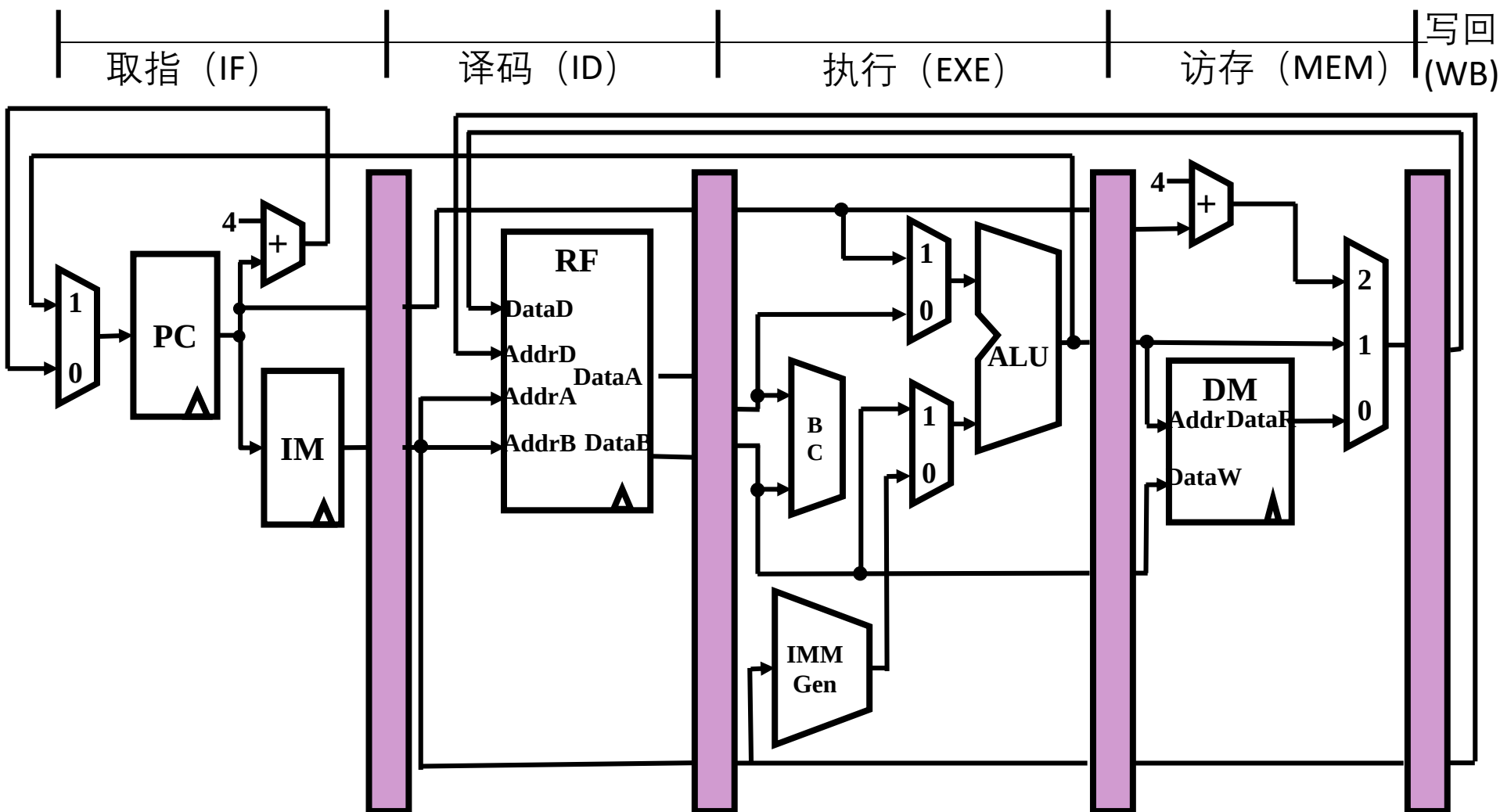
□ 硬件调试



五级流水线RISC-V处理器 回顾

理解流水线

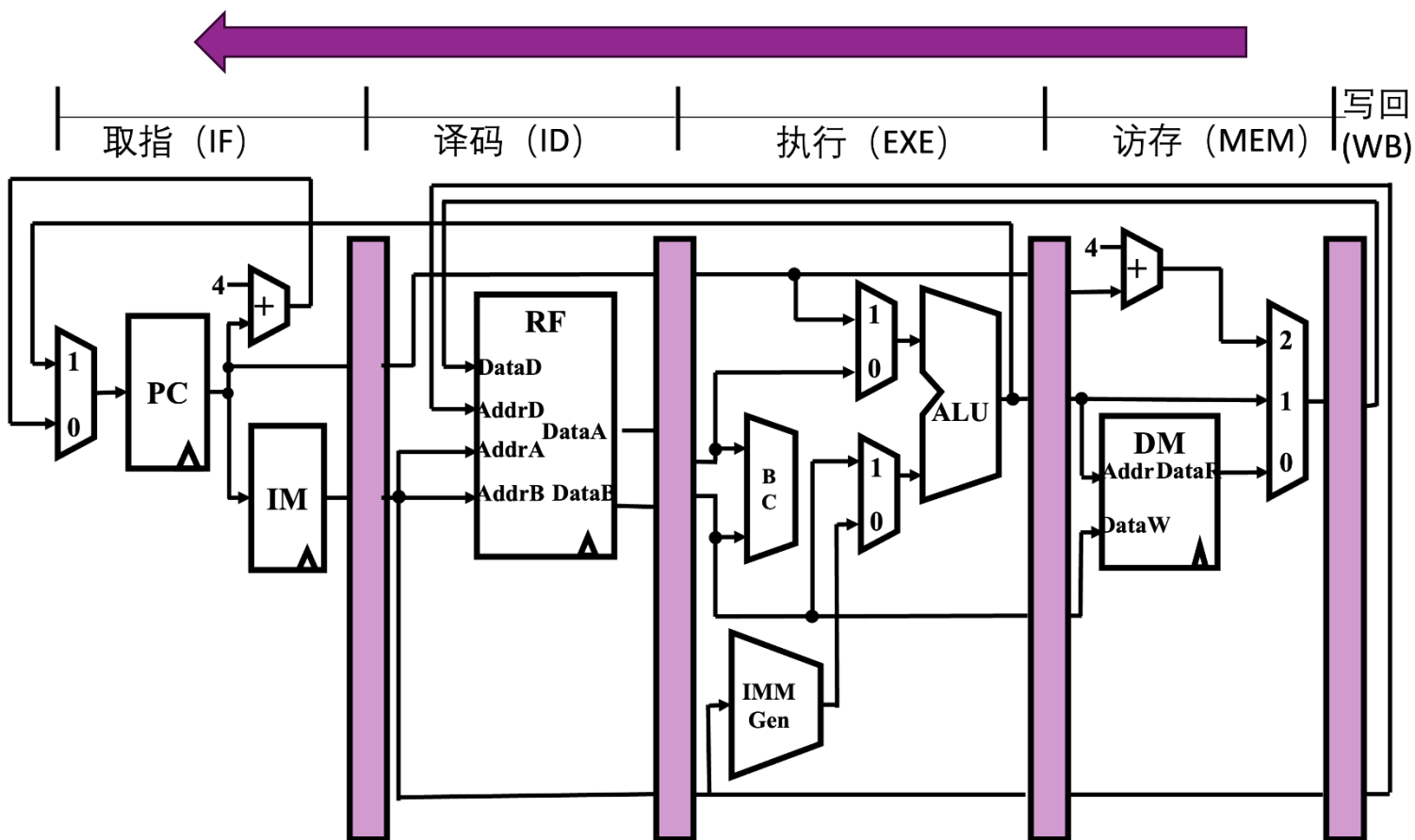
流水线CPU中的大部分控制信号由ID段生成，一段一段传到后续阶段进行处理。



设计流水线

□ 逆向考虑

- 从WB阶段开始，倒着考虑每一个流水段需要的控制信号
- 保证在设计信号的时候不会漏掉信号



WB阶段-以addi指令为例

- 对于 addi 指令的 WB 阶段，需要做的是将 alu 计算得到的 result 写回寄存器中，因此 WB 阶段需要的信号如下：
 - `rf_wen = 1`: 表示是否写寄存器（ID 段在解析指令后给出）
 - `rf_wdata = alu_result`: 写寄存器的数据（EXE 段计算后得出）
 - `rf_waddr = rd`: 写寄存器的编号（ID 段在解析指令后给出）
- 具体实现方式可以是将 `mem_wb_reg` 的对应信号直接引到 `regfile` 的写端口上。

MEM阶段-以addi指令为例

- 对于 addi 指令的 MEM 阶段，不需要做任何事情，因此需要的信号如下：
 - $\text{mem_en} = 0$: 表示是否有内存操作（ID 段在解析指令后给出）
- 对于load/store指令，MEM段需要其他相应的控制信号，需要进行思考整理

EXE阶段-以addi指令为例

- ❑ 对于 addi 指令的 EXE 阶段，需要生成立即数，同时在 ALU 中进行加法运算，因此需要的信号如下：
 - rf_rdata_a/rf_rdata_b: 表示寄存器读取 rs1,rs2 地址的结果（ID 段在读取寄存器后给出）
 - inst: 指令，用于生成立即数（IF 段在读取 SRAM 后给出）
 - imm_type = I: 立即数种类（ID 段在解析指令后给出）
 - alu_op = ADD: alu 运算的种类（ID 段在解析指令后给出）
 - use_rs2 = false: 表示第二个 operand 是立即数还是 rs2 的读取结果（ID 段在解析指令后给出）
- ❑ 同时需要给出 rf_wdata = alu_result，以供 WB 阶段使用

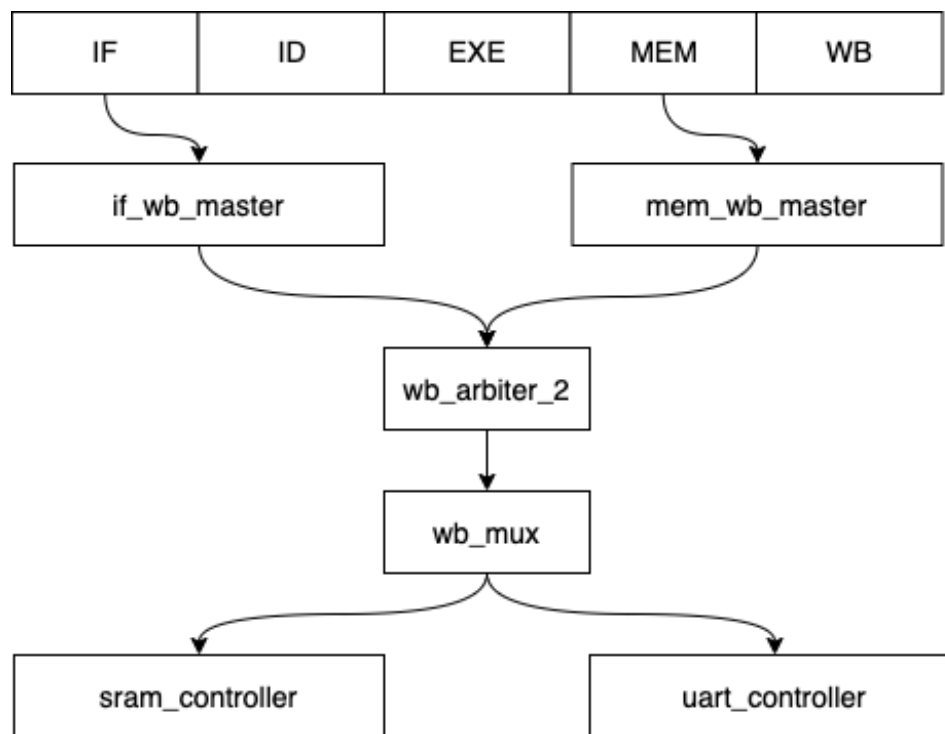
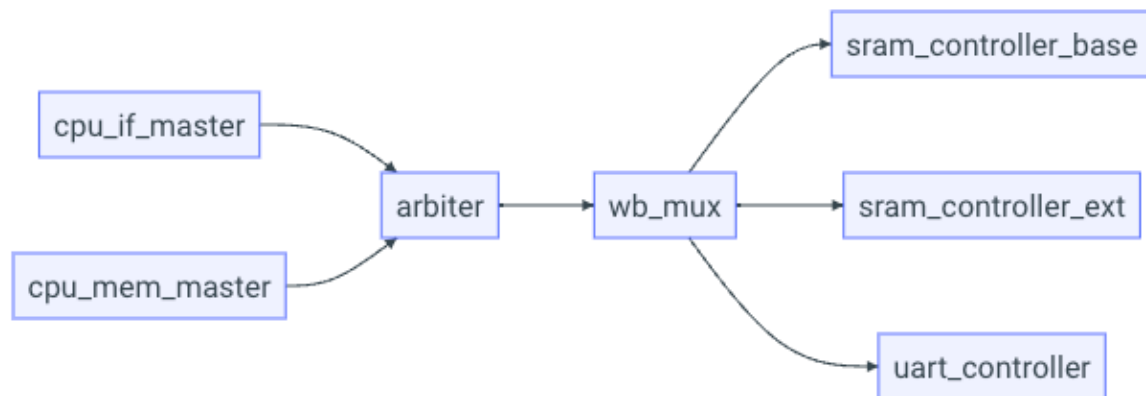
ID阶段-以addi指令为例

- ❑ 这个阶段对于所有指令都需要生成对应的控制信号，同时完成寄存器的读取。对于 addi 指令，需要的信号和对应的值如下：
 - `inst = inst`
 - `rf_raddr_a = rs1 segment of instruction`
 - `rf_raddr_b = rs2 segment of instruction`
 - ...
- ❑ 注意到需要进行可能的冲突处理，因此，若位于 EXE/MEM/WB 阶段的指令要写寄存器，且 ID 阶段的指令需要读这个寄存器，则不应该让 ID 阶段的指令进入下一阶段 (EXE)。否则这里将会读到错误 (旧) 的寄存器数据，为此需要插入气泡或引入数据旁路

IF阶段-以addi指令为例

- ❑ 这个阶段对于所有指令都需要进行 SRAM 的读取，从而获取对应的指令。
- ❑ 在 IF 阶段还要完成 pc 的更新，由于这里只介绍 addi 指令的实现，因此直接让 $pc \leq pc + 4$ 即可。但由于还有其他的指令会修改 pc，因此建议大家将 pc_mux 实现成一个单独的模块，通过控制信号控制 pc 的更新。

SOC设计



气泡和冲刷流水线

- 流水线上的冲突、访存的等待...
- 针对流水段寄存器的信号：
 - `stall_i`: 表示下个周期将忽略输入，维持输出不变
 - `bubble_i`: 表示下个周期清空该寄存器（变为气泡），输出也为气泡
- 流水段寄存器中的信号决定了这个流水段的行为
- 想象流水线的时空图
- 由controller来生成这些信号

气泡的例子

□ ID阶段发现数据冲突，需要阻塞流水线

IF	ID	EXE	MEM	WB
A	B	C	D	E

□ 则下个周期流水线中的指令应该为

IF	ID	EXE	MEM	WB
A	B	bub	C	D

寄存器	stall_i	bubble_i
if_id	1	0
id_ex	0	1
ex_mem	0	0
mem_wb	0	0



进阶功能提示

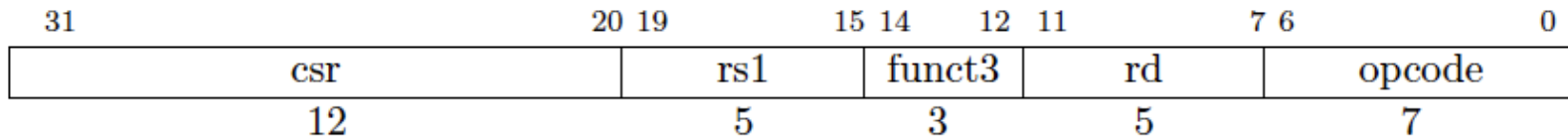
中断异常实现-step1

□ 实现CSR寄存器和其读写指令

- CSR寄存器以稀疏的方式实现，需要单独实现每个CSR寄存器。
- 为了避免处理 RAW 冲突，我们可以将 CSR 寄存器的读和写放在同一段中。
 - 若全部放在 MEM 段，则需要注意 CSR 的读取指令获得数据的时间与 load 类指令相同，需要参考 load-relate 类数据冲突进行额外处理。
 - 若全部放在 EX 段，则需要注意此时 EX 段会改变 CSR 寄存器的值，可能带来副作用，需要在异常处理中进行特殊处理。

□ 尝试在监控程序中使用CSR读写指令，检查实现的正确性

CSR指令



- ❑ CSRRW: $rd \leftarrow CSR, CSR \leftarrow rs1$
- ❑ CSRRS: $rd \leftarrow CSR$, set CSR bit by rs1
- ❑ CSRRC: $rd \leftarrow CSR$, clear CSR bit by rs1
- ❑ CSRRWI, CSRRSI, CSRRCI: 对应指令的立即数版本

重要的CSR寄存器

Machine Status Register (mstatus)

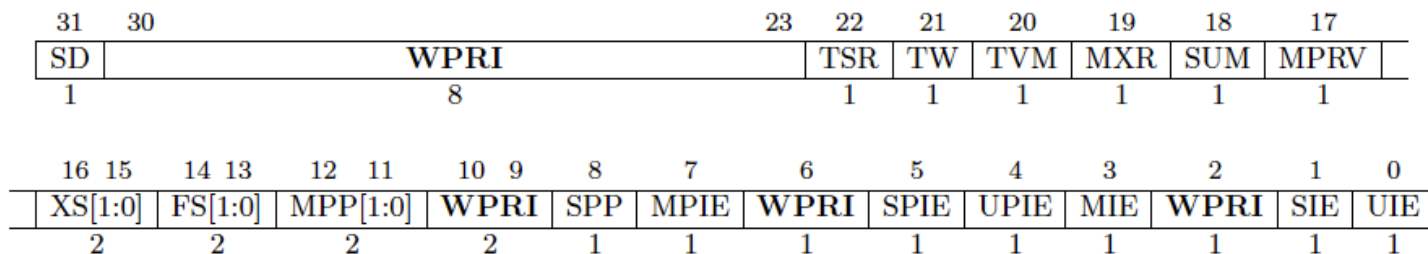


Figure 3.6: Machine-mode status register (mstatus) for RV32.

- xPP: Previous privilege mode
- xPIE: The interrupt-enable bit active prior to the trap
- 异常发生时，更新xPP和xPIE
- 通过xRET返回时，特权态切换至xPP，xIE赋值为xPIE

重要的CSR寄存器

- ❑ Machine Trap-Vector Base-Address Register (mtvec): 异常处理程序入口基地址
- ❑ Machine Exception Program Counter (mepc): 保存出现异常的指令pc, 从异常处理返回时和xRET配合使用
- ❑ Machine Cause Register (mcause): 保存异常号

中断异常实现-step2

□ 基本的异常处理

- RISC-V要求精确异常，因此需要保证在异常发生之后的指令不会改变 CPU 的“状态”而带来副作用。

流水线段	可能的副作用
IF	仅读取内存，无副作用
ID	仅读取寄存器，无副作用
EX	跳转，修改 PC
MEM	写内存，写 CSR
WB	写通用寄存器

中断异常实现-step2

- ❑ 在 EX 段直接进行异常处理。但这样的方式有一个潜在的坏处——MEM 段可能会发生缺页异常，需要在 EX 段提前查询页表，探测可能的缺页异常进行处理。
- ❑ 另一种方式是在 MEM 段进行异常处理，并且当 EX 段需要跳转时，先等待 MEM 段宣布没有异常之后，再跳转，这样也可以保证 CPU 正确性。
- ❑ 实现完成后，尝试在监控程序的用户程序中使用 `ecall` 指令访问串口输出信息。

中断实现

- ❑ 中断的处理不需要在发生中断的第一个周期就立即进行处理，可以选择以下面的方式实现中断的处理：
 - IF 段取指令时 / ID 段译码时，检查当前是否满足中断发生的条件，并在发生中断时给指令带上异常标记
 - 进入 EX 阶段后，跳过该指令在 EX 段的执行。
 - 在异常处理阶段接受中断，修改对应的 CSR 寄存器。

重要的CSR寄存器

□ Machine Interrupt Registers (mip and mie)

MXLEN-1	12	11	10	9	8	7	6	5	4	3	2	1	0
WPRI	MEIP	WPRI	SEIP	UEIP	MTIP	WPRI	STIP	UTIP	MSIP	WPRI	SSIP	USIP	
MXLEN-12	1	1	1	1	1	1	1	1	1	1	1	1	

Figure 3.11: Machine interrupt-pending register (mip).

MXLEN-1	12	11	10	9	8	7	6	5	4	3	2	1	0
WPRI	MEIE	WPRI	SEIE	UEIE	MTIE	WPRI	STIE	UTIE	MSIE	WPRI	SSIE	USIE	
MXLEN-12	1	1	1	1	1	1	1	1	1	1	1	1	

Figure 3.12: Machine interrupt-enable register (mie).

时钟中断实现

□ 实现mtime和mtimecmp两个CSR寄存器

- 这两个寄存器不通过CSR读写指令读写，通过load/store的方式进行处理
- 参考串口，给 mtime 和 mtimecmp 实现单独的 wishbone slave (MMIO)
- 在 MEM 段访问 wishbone 总线之前进行特判，重定向对这两个寄存器的请求

页表实现提示

- ❑ Supervisor Address Translation and Protection (satp) Register
- ❑ Holds the physical page number (PPN) of the root page table

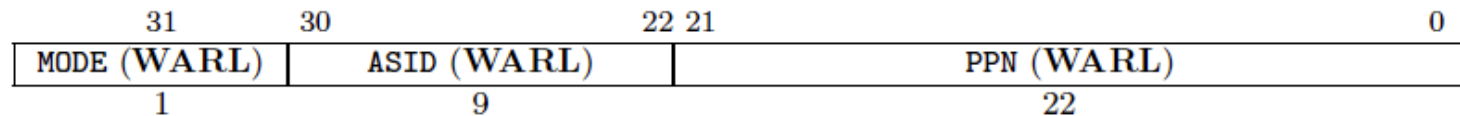


Figure 4.11: RV32 Supervisor address translation and protection register `satp`.

虚拟地址到物理地址的转换

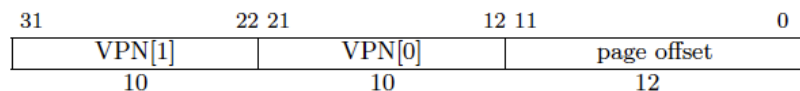


Figure 4.13: Sv32 virtual address.

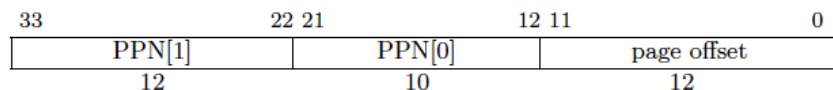


Figure 4.14: Sv32 physical address.

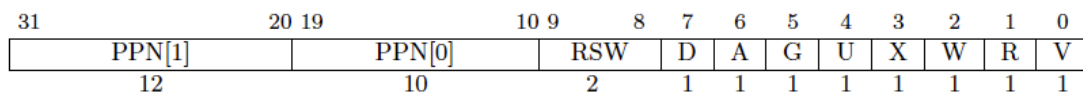
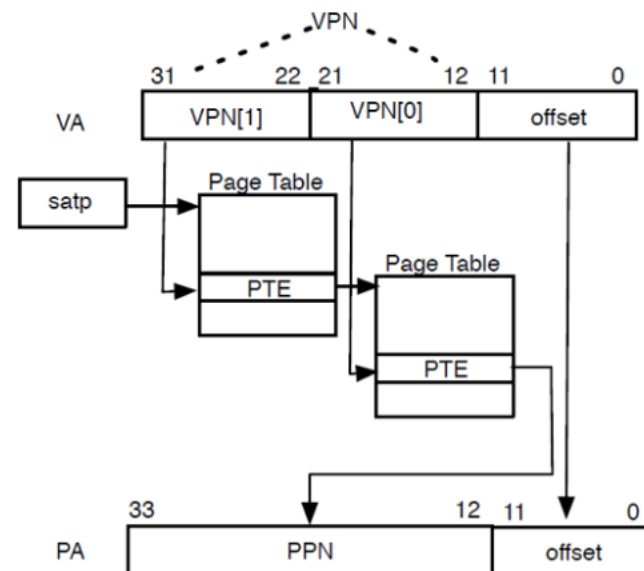


Figure 4.15: Sv32 page table entry.



页表实现提示

- ❑ 在IF段和MEM段的访存状态机中，查询页表，检查权限。
- ❑ 如果在 EX 段处理异常，则需要 EX 段提前查询页表，检查缺页异常。这可能需要一个额外的 wishbone master。
- ❑ 如果在 MEM 段处理异常，则需要检查完页表之前，阻拦潜在的 EX 段跳转和 CSR 写入。
- ❑ 拼出来的 34 位物理地址可以直接去掉最高的两位当作 32 位地址进行使用。
- ❑ 不需要实现 RSW, D, A, G 位的相关内容。

页表实现测试

- 实现完成之后，将用户程序写入 0x80100000 的物理地址，并在 G 命令中指定运行地址为 0x0 应该可以正常执行用户程序。



监控程序运行流程

编译监控程序

- ❑ Linux: 使用make命令
- ❑ Windows: 使用`编译 Kernel 并启动模拟器.cmd`
- ❑ 修改编译参数得到不同版本的监控程序
- ❑ 得到`kernel.bin`

上传设计文件和监控程序

- ❑ 在ThinPAD-Cloud云平台上传设计的bitstream
- ❑ 使用RAM读写功能将监控程序对应的bin文件写入sram中

Flash&RAM 读写 使用本功能将重置FPGA

存储选择 BaseRAM ▾ 容量 4MB

起始地址 0x 0 (Byte)
起始地址应当按16位对齐（即为偶数）

读取数据 0x 读取长度 (Byte) 查看 下载
读取长度应当按16位对齐（即为偶数）

写入数据 选择文件 未选择任何文件 写入
写入长度为数据文件大小，按16位对齐（即为偶数）

启动终端程序连接云平台

□ 打开云平台串口，找到下方网络转发TCP端口号



The screenshot shows a configuration panel for a serial port connection. It includes two dropdown menus for '数据位' (Data Bits) set to 8 and '停止位' (Stop Bits) set to 1. Below these, the '网络转发' (Network Forwarding) section displays the IP address '166.111.226.111:37601' in red text, with the note '通过 TCP 连接访问串口' (Access serial port via TCP connection) below it. At the bottom, there are three blue buttons: '发送' (Send), '终端模式' (Terminal Mode), and '关闭串口' (Close Serial Port).

□ 修改终端程序启动参数中的TCP端口参数，连接云平台`python3 term.py -t 127.0.0.1:6666`

运行性能测试程序

- 编译监控程序时加参数 show-utest可以得到内置测试程序列表

```
1:80001000 <UTEST_SIMPLE>:  
2:80001008 <UTEST_1PTB>:  
3:80001024 <UTEST_2DCT>:  
4:80001064 <UTEST_3CCT>:  
5:80001080 <UTEST_4MDCT>:  
6:800010a8 <UTEST_CRYPTONIGHT>:
```

- 通过终端连接到云平台上的处理器设计后，可以通过G命令运行对应地址的程序，记录程序运行的时间

更多进阶功能

- ❑ 其他更多进阶功能的提示详见实验指导书“进阶功能提示”一节
- ❑ <https://lab.cs.tsinghua.edu.cn/cod-lab-docs/labs/ex2/hint/>



常见问题

常见的编码问题-多源驱动

❑ Critical Warning :
[Synth 8-3352] multi-
driven net
<signal_name> with
1st driver pin
'<pin_name1>' [xxx.v.3]

❑ 问题原因

- 同一触发器（reg）存在多个驱动源

❑ 解决方法

- 一个reg只能在一个always块中赋值

```
always @(posedge clk) begin
    if (xxx) begin
        s <= 1;
    end
end
always @(posedge clk) begin
    if(yyy) begin
        s <= 0;
    end
end
```

常见编码问题-电平锁存器

❑ WARNING: [Synth 8-327] inferring latch for variable 's_reg' [*.v:12]

❑ 问题原因

- 组合逻辑敏感信号列表不全，空分支

❑ 解决方法

- 描述组合逻辑时用 always@(*), 并且保证所有分支均被覆盖

```
always @(a) begin
    if (a) begin
        s <= 1;
    end else if(b) begin
        s <= 0;
    end
end
```

```
always @(*) begin
    if (a) begin
        s <= 1;
    end else if(b) begin
        s <= 0;
    end else begin
        s <= 0;
    end
end
```

s	a	b
1	1	x
0	0	1
0	0	0

常见编码问题-跨时钟域

□ 问题原因

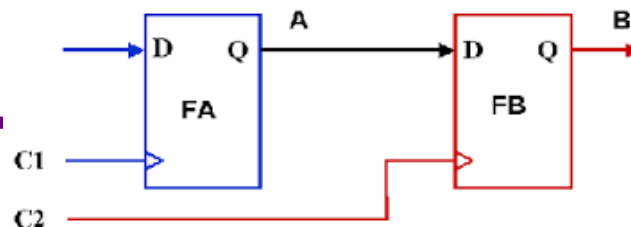
- 捕获一个来自不同的时钟域的信号时，建立时间不满足，出现亚稳态

□ 解决方法

- 使用两级触发器，构造一个同步器，消除亚稳态

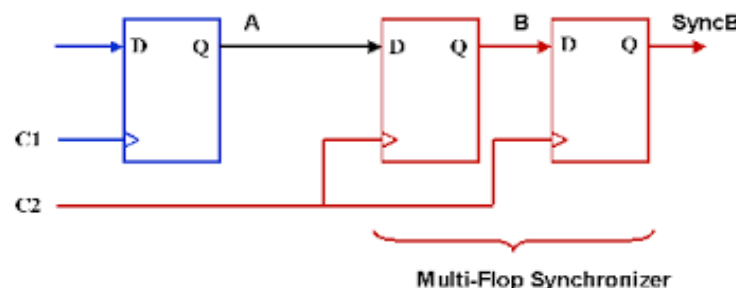
□ 扩展阅读：

<https://www.fpga4fun.com/CrossClockDomain.html>



```
reg in, a, b;  
always @(posedge c1) begin  
    a <= in;  
end  
always @(posedge c2) begin  
    b <= a;  
end
```

```
reg in, a, b, sync_b;  
always @(posedge c1) begin  
    a <= in;  
end  
always @(posedge c2) begin  
    b <= a;  
    sync_b <= b;  
end
```



其他问题

- 在线测评不通过，可能是什么现象？
- 仿真和上板行为不一致，可能是什么现象？
- 出现了xxx行为，可能是什么现象？

- 建议
 - 先根据实验文档“错误速查检查单”进行自查
 - 消除warning和所有X
 - 在FAQ中进行检索是否有类似问题

谢谢